Stealth Addresses and Ring Signatures for Transaction Privacy

Pars Network Technical Whitepaper PNP-007

Cyrus Pahlavi, Pars Team research@pars.network Version 1.0 — February 2026

Abstract

We present the Pars transaction privacy system, combining stealth addresses for recipient anonymity with ring signatures for sender anonymity on the Pars L2 rollup. Stealth addresses use the Dual-Key Stealth Address Protocol (DKSAP) adapted from EIP-5564, enabling senders to generate one-time addresses from a recipient's stealth meta-address without on-chain interaction. Ring signatures employ a linkable spontaneous anonymous group (LSAG) scheme that hides the true signer among a set of decoys while preventing double-spending through key images. We integrate Pedersen commitments for amount hiding with Bulletproofs range proofs [1] to ensure transaction validity without revealing values. The combined system achieves sender anonymity set size of 11 (1 real + 10 decoys), recipient unlinkability via ephemeral one-time addresses, and amount confidentiality. On-chain verification of a ring-signature transaction costs approximately 450 000 gas on the Pars L2, with a transaction size of 2.8 KB for a standard ring size of 11.

Keywords: stealth addresses, ring signatures, transaction privacy, one-time addresses, linkability prevention, confidential transactions

Contents

1 Introduction **4**
1.1 Privacy Properties . . . 4
1.2 Contributions . . . 4

2 Background **4**
2.1 Stealth Addresses . . . 4
2.2 Ring Signatures . . . 4
2.3 Confidential Transactions . . . 4

3 Stealth Address Protocol **5**
3.1 Key Generation . . . 5
3.2 Payment Protocol . . . 5
3.3 Announcement Contract . . . 6

4 Ring Signature Scheme **6**
4.1 LSAG Construction . . . 6
4.2 Ring Member Selection . . . 7

5 Confidential Transactions **8**
5.1 Pedersen Commitments . . . 8
5.2 Balance Verification . . . 8
5.3 Range Proofs . . . 8

6 Combined Transaction Format **8**
6.1 Private Transaction Structure . . . 8
6.2 Transaction Size Analysis . . . 9

7 Linkability Prevention **9**
7.1 Transaction Graph Analysis . . . 9
7.2 Timing Analysis Resistance . . . 10

8 View Key Mechanism **10**
8.1 Selective Auditability . . . 10
8.2 Compliance Bridge . . . 11

9 On-Chain Verification **12**
9.1 Verification Contract 12
9.2 Gas Cost Analysis 12

10 Security Analysis **12**
10.1 Anonymity Set 12
10.2 Known Attacks and Mitigations 13
10.3 Formal Properties 13

11 Related Work **13**

12 Conclusion **14**

A Hash-to-Point Function **15**

B Decoy Selection Distribution **15**

1 Introduction

Public blockchains are pseudonymous, not anonymous. Every transaction on the Pars L2 reveals the sender address, recipient address, and transfer amount. Chain analysis firms can link pseudonymous addresses to real identities through exchange records, IP addresses, and transaction patterns [2].

For diaspora communities, this transparency is dangerous. A remittance from a diaspora member to a family member within Iran can be traced, potentially exposing both parties to state retaliation [3]. Governance token transfers can reveal political affiliations. Even benign transactions accumulate into a profile that enables surveillance.

1.1 Privacy Properties

We target three privacy properties:

1. **Sender Anonymity:** The true sender is hidden among a set of plausible alternatives (ring members).
2. **Recipient Unlinkability:** Each payment to the same recipient uses a fresh one-time address, unlinkable to other payments or to the recipient's public identity.
3. **Amount Confidentiality:** Transaction amounts are hidden behind cryptographic commitments, verifiable without disclosure.

1.2 Contributions

- Adaptation of DKSAP stealth addresses for the Pars L2 EVM.
- LSAG ring signature scheme with EVM-optimised verification.
- Pedersen commitment scheme with Bulletproofs range proofs.
- View key mechanism for selective auditability.
- Gas-optimised implementation for L2 deployment.

2 Background

2.1 Stealth Addresses

Stealth addresses were introduced by Todd [4] and formalised in EIP-5564 [5]. The core idea: a recipient publishes a *stealth meta-address* from which senders derive fresh one-time addresses for each payment.

2.2 Ring Signatures

Ring signatures, introduced by Rivest, Shamir, and Tauman [6], allow a signer to sign a message on behalf of a group (ring) without revealing which member signed. CryptoNote protocols [7] adapted ring signatures for transaction privacy. Monero [8] refined the approach with linkable ring signatures and RingCT.

2.3 Confidential Transactions

Pedersen commitments [9] hide transaction amounts: $C = g^v h^r$ commits to value v with blinding factor r. The homomorphic property ($C_1 \cdot C_2 = g^{v_1+v_2} h^{r_1+r_2}$) enables balance verification without value disclosure. Bulletproofs [1] prove $v \in [0, 2^{64})$ without revealing v.

3 Stealth Address Protocol

3.1 Key Generation

Definition 3.1 (Stealth Meta-Address)**.** *A stealth meta-address consists of two public keys:*

- *Spending public key:* $K_s = k_s G$ *(controls funds)*
- *Viewing public key:* $K_v = k_v G$ *(scans for payments)*

Published as: `pars:st:`$\langle K_s \rangle$`:`$\langle K_v \rangle$

The separation of spending and viewing keys enables delegation: a user can share k_v with a trusted third party (e.g., an accountant) to scan for incoming payments without granting spending authority.

3.2 Payment Protocol

Algorithm 1 Stealth Address Payment (Sender)

Require: Recipient's stealth meta-address (K_s, K_v), amount v

Generate ephemeral key pair: $(r, R = rG)$
Compute shared secret: $S = r \cdot K_v = rk_vG$
Derive stealth key offset: $t = H(S \| \text{nonce})$
Compute one-time stealth address:
  $P = K_s + tG$ ▷ recipient's one-time PK
Compute stealth Ethereum address:
  $\text{addr} = \text{keccak256}(P)[12:]$
Create transaction to addr with value v
Publish ephemeral public key R in an announcement contract

Algorithm 2 Stealth Address Scanning (Recipient)

Require: Viewing key k_v, spending key k_s, announcements $\{R_i\}$

for all announced R_i **do**
  $S_i = k_v \cdot R_i$ ▷ shared secret
  $t_i = H(S_i \| \text{nonce}_i)$
  $P_i = K_s + t_i G$
  $\text{addr}_i = \text{keccak256}(P_i)[12:]$
  if addr_i has balance on L2 **then**
    Stealth private key: $p_i = k_s + t_i$
    Record: payment found at addr_i with key p_i
  end if
end for

Theorem 3.1 (Recipient Unlinkability)**.** *Two stealth addresses* $\text{addr}_1, \text{addr}_2$ *generated for the same recipient are computationally unlinkable without the viewing key* k_v*.*

Proof. addr_1 derives from $R_1, t_1 = H(r_1 k_v G \| n_1)$ and addr_2 from $R_2, t_2 = H(r_2 k_v G \| n_2)$. Linking requires computing t_1, t_2 from R_1, R_2, which requires solving CDH (computing $r_i k_v G$ from $R_i = r_i G$ and $K_v = k_v G$) or knowing k_v. Under CDH hardness on the secp256k1 curve, this is infeasible. □

3.3 Announcement Contract

Listing 1: Stealth Announcement Contract

```
contract StealthAnnouncer {
    event Announcement(
        uint256 indexed schemeId,
        address indexed stealthAddr,
        address indexed caller,
        bytes ephemeralPubKey,
        bytes metadata
    );

    function announce(
        uint256 schemeId,
        address stealthAddr,
        bytes calldata ephPubKey,
        bytes calldata metadata
    ) external {
        emit Announcement(
            schemeId,
            stealthAddr,
            msg.sender,
            ephPubKey,
            metadata
        );
    }
}
```

4 Ring Signature Scheme

4.1 LSAG Construction

We use the Linkable Spontaneous Anonymous Group (LSAG) scheme [10]:

Definition 4.1 (LSAG Signature)**.** *Given ring $\mathcal{R} = \{P_0, \ldots, P_{n-1}\}$, signer index π, and message m:*

Key Image: *$I = k_\pi H_p(P_\pi)$ where H_p maps to a curve point and k_π is the signer's secret key.*

The key image I is deterministic for a given key pair, enabling double-spend detection while preserving signer anonymity.

Algorithm 3 LSAG Ring Signature Generation

Require: Ring $\mathcal{R} = \{P_i\}_{i=0}^{n-1}$, signer key k_π, message m

$I \leftarrow k_\pi \cdot H_p(P_\pi)$ ▷ key image
Choose random $\alpha \leftarrow \mathbb{Z}_q$
$c_{\pi+1} \leftarrow H(m \| \alpha G \| \alpha H_p(P_\pi))$
for $i = \pi + 1, \ldots, n-1, 0, \ldots, \pi - 1$ **do**
    Choose random $s_i \leftarrow \mathbb{Z}_q$
    $c_{i+1} \leftarrow H(m \| s_i G + c_i P_i \| s_i H_p(P_i) + c_i I)$
end for
$s_\pi \leftarrow \alpha - c_\pi k_\pi \bmod q$
return $\sigma = (c_0, s_0, \ldots, s_{n-1}, I)$

Algorithm 4 LSAG Ring Signature Verification

Require: Ring $\mathcal{R}$, message m, signature $\sigma = (c_0, \{s_i\}, I)$
1: Check $I \notin$ key image set (double-spend check)
2: **for** $i = 0, \ldots, n-1$ **do**
3: $\quad c'_{i+1} \leftarrow H(m \| s_i G + c_i P_i \| s_i H_p(P_i) + c_i I)$
4: **end for**
5: Accept iff $c'_0 = c_0$
6: If accepted, add I to key image set

Theorem 4.1 (Anonymity)**.** *Given a valid LSAG signature, no PPT adversary can determine the true signer's index π with probability better than $1/n$, under the DDH assumption.*

Proof. By DDH, the verification equations $s_i G + c_i P_i$ and $s_i H_p(P_i) + c_i I$ are indistinguishable across all indices. Each index i produces a valid "chain" of commitments. Without knowing which s_i was derived from $\alpha - c_i k_i$ (vs. chosen randomly), the adversary cannot identify π. □

Theorem 4.2 (Linkability)**.** *Two signatures produced by the same private key generate the same key image I, enabling double-spend detection.*

Proof. $I = k \cdot H_p(P)$ is deterministic in k and $P = kG$. Any two signatures by key k produce the same I regardless of ring composition. Different keys produce different images (collision implies solving DLP on $H_p(P)$). □

4.2 Ring Member Selection

Ring members (decoys) are selected from recent L2 transaction outputs:

Listing 2: Ring Member Selection

```go
type RingSelector struct {
    chainDB  ChainDB
    ringSize int // default 11
}

func (s *RingSelector) SelectRing(
    realOutput *TxOutput,
) ([]*TxOutput, int, error) {
    // Get recent outputs for decoy pool
    pool, err := s.chainDB.RecentOutputs(
        1000, // last 1000 blocks
    )
    if err != nil {
        return nil, 0, fmt.Errorf(
            "pool: %w", err,
        )
    }

    // Select decoys with similar age
    // and amount distribution
    decoys := selectByDistribution(
        pool, realOutput, s.ringSize-1,
    )

    // Insert real output at random position
    ring := make([]*TxOutput, s.ringSize)
    pi := rand.Intn(s.ringSize)
    j := 0
```

```
    for i := 0; i < s.ringSize; i++ {
        if i == pi {
            ring[i] = realOutput
        } else {
            ring[i] = decoys[j]
            j++
        }
    }
    return ring, pi, nil
}
```

5 Confidential Transactions

5.1 Pedersen Commitments

Definition 5.1 (Pedersen Commitment)**.** *For value v and blinding factor r: $C = vG + rH$ where G, H are independent generators of the curve group and the discrete log relation between them is unknown.*

5.2 Balance Verification

For a transaction with inputs $\{C_{\text{in},j}\}$ and outputs $\{C_{\text{out},k}\}$:

$$\sum_j C_{\text{in},j} - \sum_k C_{\text{out},k} = (\sum_j v_j - \sum_k v_k)G + (\sum_j r_j - \sum_k r_k)H$$

If values balance ($\sum v_{\text{in}} = \sum v_{\text{out}} + \text{fee}$), the result is a commitment to the fee with a known blinding factor, which the transaction includes for verification.

5.3 Range Proofs

Each output commitment requires a range proof to prevent negative values:

Definition 5.2 (Bulletproof Range Proof)**.** *Prove $v \in [0, 2^{64})$ for commitment $C = vG + rH$ without revealing v or r. Proof size: $2\lceil \log_2(64) \rceil + 9 = 21$ group elements (≈ 672 bytes).*

For multi-output transactions, aggregated Bulletproofs reduce proof size:

Table 1: Bulletproof Sizes (64-bit range)

Outputs	Proof Size
1	672 B
2	736 B
4	800 B
8	864 B

6 Combined Transaction Format

6.1 Private Transaction Structure

Listing 3: Private Transaction Format

```go
type PrivateTransaction struct {
    // Ring signature over inputs
    Ring        []*PublicKey   // ring members
    KeyImage    *Point         // linkable key image
    RingSig     *LsagSignature
    // Stealth outputs
    Outputs     []StealthOutput
    // Fee (plaintext for sequencer)
    Fee         uint64
    // Ephemeral keys for each output
    EphKeys     []*PublicKey
}

type StealthOutput struct {
    StealthAddr common.Address
    Commitment  *Point   // Pedersen commitment
    RangeProof  []byte   // Bulletproof
    EncAmount   []byte   // Encrypted amount
                         // (for recipient only)
}

type LsagSignature struct {
    C0      *big.Int
    S       []*big.Int   // one per ring member
    Image   *Point       // key image
}
```

6.2 Transaction Size Analysis

Table 2: Transaction Size Breakdown (ring=11, 2 outputs)

Component	**Size**
Ring public keys (11 × 33 B)	363 B
Key image (33 B)	33 B
Ring signature scalars (11 × 32 B)	352 B
c_0 (32 B)	32 B
Stealth addresses (2 × 20 B)	40 B
Commitments (2 × 33 B)	66 B
Range proof (aggregated, 2 outputs)	736 B
Encrypted amounts (2 × 32 B)	64 B
Ephemeral keys (2 × 33 B)	66 B
Fee + metadata	40 B
Total	**1792 B**

7 Linkability Prevention

7.1 Transaction Graph Analysis

Beyond ring signatures and stealth addresses, we employ additional measures against graph analysis:

1. **Uniform Ring Selection:** Decoys are selected with a distribution that matches real spending patterns (gamma distribution over output age).
2. **Minimum Ring Size:** All transactions must use ring size ≥ 11, ensuring a minimum anonymity set.
3. **Churning:** Users can periodically send funds to themselves via stealth addresses, refreshing the output set and breaking temporal correlations.
4. **Decoy Output Flooding:** The protocol periodically creates dummy outputs (funded by the treasury) to expand the decoy pool.

7.2 Timing Analysis Resistance

Transactions are submitted through the session layer (PNP-003) with randomised delays:

Listing 4: Timing Obfuscation

```go
func (w *PrivateWallet) SubmitTx(
    ctx context.Context,
    tx *PrivateTransaction,
) error {
    // Add random delay (Poisson distributed)
    delay := poissonDelay(
        30 * time.Second, // mean
    )
    select {
    case <-time.After(delay):
    case <-ctx.Done():
        return ctx.Err()
    }
    // Submit via session layer
    return w.session.Submit(ctx, tx)
}

func poissonDelay(mean time.Duration) time.Duration {
    // Exponential distribution (memoryless)
    lambda := 1.0 / float64(mean)
    u := rand.Float64()
    return time.Duration(
        -math.Log(1-u) / lambda,
    )
}
```

8 View Key Mechanism

8.1 Selective Auditability

The viewing key k_v enables selective disclosure to auditors:

Algorithm 5 View Key Audit

Require: Auditor with viewing key k_v, announcement log

for all announcements (R_i, addr_i) **do**
  $S_i = k_v \cdot R_i$
  $t_i = H(S_i \| n_i)$
  $P_i = K_s + t_i G$
  $\text{check}_i = \text{keccak256}(P_i)[12:]$
  if $\text{check}_i = \text{addr}_i$ **then**
    Payment to this recipient found
    Decrypt amount: $v_i = \text{Dec}(H(S_i), \text{encAmt}_i)$
    Record $(v_i, \text{addr}_i, \text{timestamp}_i)$
  end if
end for
Auditor sees all incoming payments and amounts
Auditor cannot spend (no k_s)
Auditor cannot see outgoing transactions (no ring key)

8.2 Compliance Bridge

For jurisdictions requiring transaction reporting:

Listing 5: Compliance View Key

```go
type ComplianceReport struct {
    Period     TimeRange
    Incoming   []IncomingPayment
    TotalIn    *big.Int
    // Outgoing not available via view key
    // (requires spending key cooperation)
}

func GenerateReport(
    viewKey *ViewKey,
    period TimeRange,
    announcements []Announcement,
) (*ComplianceReport, error) {
    report := &ComplianceReport{
        Period: period,
    }
    for _, ann := range announcements {
        if !period.Contains(ann.Timestamp) {
            continue
        }
        payment, ok := viewKey.Scan(ann)
        if ok {
            report.Incoming = append(
                report.Incoming, payment,
            )
            report.TotalIn.Add(
                report.TotalIn, payment.Amount,
            )
        }
    }
    return report, nil
}
```

9 On-Chain Verification

9.1 Verification Contract

Listing 6: Ring Signature Verifier

```
contract RingVerifier {
    // Key images seen (double-spend prevention)
    mapping(bytes32 => bool) public spent;

    function verifyRingTx(
        bytes calldata ringPKs,
        bytes calldata ringSig,
        bytes32 keyImage,
        bytes calldata outputs,
        bytes calldata rangeProofs
    ) external returns (bool) {
        // 1. Check key image not spent
        require(!spent[keyImage], "Double␣spend");

        // 2. Verify LSAG ring signature
        require(
            _verifyLSAG(
                ringPKs, ringSig, keyImage
            ), "Bad␣ring␣sig"
        );

        // 3. Verify range proofs
        require(
            _verifyBulletproofs(
                outputs, rangeProofs
            ), "Bad␣range␣proof"
        );

        // 4. Verify balance
        require(
            _verifyBalance(outputs),
            "Unbalanced"
        );

        // 5. Mark key image as spent
        spent[keyImage] = true;

        return true;
    }
}
```

9.2 Gas Cost Analysis

10 Security Analysis

10.1 Anonymity Set

Proposition 10.1 (Effective Anonymity). *With ring size $n = 11$ and a decoy pool of 10 000 recent outputs, the probability that an adversary correctly identifies the true signer is $1/11 \approx 9.1\%$ for a single transaction. Over k transactions, if ring members are selected independently, the*

Table 3: Verification Gas Costs (ring=11)

Operation	**Gas**
Key image uniqueness check	20 000
LSAG verification (11 members)	180 000
Bulletproof verification (2 out)	200 000
Balance check	15 000
Key image storage	20 000
Event emission	5 000
Total	**440 000**

intersection attack reduces the effective anonymity set to approximately $n^{1-(k-1)/k} = n^{1/k}$ *for* k *linked transactions.*

10.2 Known Attacks and Mitigations

Table 4: Known Attacks and Mitigations

Attack	**Description**	**Mitigation**
Poisoned decoys	Adversary creates outputs to narrow set	Minimum output age requirement (10 blocks)
Timing analysis	Correlate tx timing with real events	Poisson delay submission via session layer
Output merging	Trace merged inputs from same wallet	Encourage churning; minimum ring size
EBE attack	Each-bug-every tracing via age patterns	Gamma distribution for decoy selection

10.3 Formal Properties

Theorem 10.1 (Sender Anonymity)**.** *Under DDH, the LSAG scheme provides perfect signer ambiguity: the signature distribution is identical regardless of which ring member signed.*

Theorem 10.2 (Non-Forgeability)**.** *Under DLP, no PPT adversary can produce a valid LSAG signature without knowing any ring member's private key.*

Theorem 10.3 (Linkability Soundness)**.** *Two signatures from the same key produce the same key image with overwhelming probability. Two signatures from different keys produce different images with overwhelming probability.*

11 Related Work

CryptoNote [7] introduced ring signatures for cryptocurrency. Monero [8] added RingCT for amount hiding. Zcash [11] uses zk-SNARKs for full privacy. Tornado Cash [12] provides mixing on Ethereum. EIP-5564 [5] standardises stealth addresses. Umbra [13] implements stealth addresses on Ethereum.

Pars combines stealth addresses with ring signatures and confidential transactions within an L2 rollup, providing a comprehensive privacy solution integrated with the session and governance layers.

12 Conclusion

The Pars transaction privacy system provides sender anonymity via LSAG ring signatures, recipient unlinkability via stealth addresses, and amount confidentiality via Pedersen commitments with Bulletproofs. The combined system achieves ring-11 sender anonymity, one-time recipient addresses, and hidden amounts at a cost of approximately 440 000 gas and 1.8 KB per transaction on the Pars L2. View keys enable selective auditability for compliance without compromising privacy.

Transaction privacy is not a feature but a requirement for diaspora infrastructure. Without it, the pseudonymity of blockchain addresses collapses under chain analysis, exposing the very people the system is designed to protect.

References

[1] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs," *IEEE S&P*, pp. 315–334, 2018.

[2] S. Meiklejohn *et al.*, "A Fistful of Bitcoins," *IMC*, pp. 127–140, 2013.

[3] Freedom House, "Freedom on the Net: Iran," 2023.

[4] P. Todd, "Stealth Addresses," *Bitcoin Wiki*, 2014.

[5] N. Johnson *et al.*, "EIP-5564: Stealth Addresses," *Ethereum Improvement Proposals*, 2022.

[6] R. Rivest, A. Shamir, and Y. Tauman, "How to Leak a Secret," *ASIACRYPT 2001*, pp. 552–565, 2001.

[7] N. van Saberhagen, "CryptoNote v2.0," 2013.

[8] S. Noether, "Ring Signature Confidential Transactions for Monero," *MRL-0005*, 2015.

[9] T. Pedersen, "Non-Interactive Verifiable Secret Sharing," *CRYPTO '91*, pp. 129–140, 1991.

[10] J. K. Liu, V. K. Wei, and D. S. Wong, "Linkable Spontaneous Anonymous Group Signature," *ACISP 2004*, pp. 325–335, 2004.

[11] E. Ben-Sasson *et al.*, "Zerocash," *IEEE S&P*, pp. 459–474, 2014.

[12] A. Pertsev, R. Semenov, and R. Storm, "Tornado Cash Privacy Solution," 2019.

[13] Umbra Protocol, "Umbra: Stealth Payments on Ethereum," 2021.

[14] G. Maxwell, "Confidential Transactions," *Bitcoin Wiki*, 2016.

[15] E. Fujisaki and K. Suzuki, "Traceable Ring Signature," *PKC 2007*, pp. 181–200, 2007.

[16] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the Weil pairing," *ASIACRYPT 2001*, 2001.

[17] D. J. Bernstein, "Curve25519," *PKC 2006*, 2006.

[18] G. Wood, "Ethereum Yellow Paper," 2014.

[19] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Trans. Inf. Theory*, 22(6), 1976.

[20] S. Goldwasser, S. Micali, and C. Rackoff, "Knowledge Complexity," *SIAM J. Comput.*, 18(1), 1989.

[21] D. Chaum, "Untraceable electronic mail," *Commun. ACM*, 24(2), 1981.

[22] S. Nakamoto, "Bitcoin," 2008.

[23] J. Groth, "On the Size of Pairing-Based Non-interactive Arguments," *EUROCRYPT 2016*, 2016.

[24] C. P. Schnorr, "Efficient Identification and Signatures for Smart Cards," *CRYPTO '89*, 1989.

[25] M. Möser *et al.*, "An Empirical Analysis of Traceability in the Monero Blockchain," *PETS*, 2018.

[26] A. Kumar *et al.*, "A Traceability Analysis of Monero's Blockchain," *ESORICS*, 2017.

[27] D. A. Wijaya *et al.*, "Monero Ring Attack," *ACM Blockchain*, 2019.

A Hash-to-Point Function

Listing 7: Hash to Curve Point

```go
func HashToPoint(data []byte) *Point {
    // Try-and-increment method
    ctr := byte(0)
    for {
        h := sha256.Sum256(
            append(data, ctr),
        )
        x := new(big.Int).SetBytes(h[:])
        x.Mod(x, curve.P)
        // Check if x is valid x-coordinate
        y := computeY(x)
        if y != nil {
            return &Point{X: x, Y: y}
        }
        ctr++
    }
}
```

B Decoy Selection Distribution

Decoys selected with gamma distribution over output age:

Listing 8: Gamma Distribution Selection

```go
func gammaSelect(
    pool []*TxOutput,
    n int,
) []*TxOutput {
    // shape=19.28, rate=1.61
    // (fitted to empirical spending)
    dist := distuv.Gamma{
        Alpha: 19.28,
        Beta:  1.61,
    }
```

```
    selected := make([]*TxOutput, 0, n)
    for len(selected) < n {
        age := dist.Rand()
        idx := ageToIndex(pool, age)
        if idx >= 0 && !contains(selected, pool[idx]) {
            selected = append(
                selected, pool[idx],
            )
        }
    }
    return selected
}
```